

SKILL WEEK-9

1 . To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, and Test Job Listings.

Aim : To implement a simple job-management system that can create background jobs, assign job IDs, maintain their process IDs and states, list active jobs, and update job status when processes finish.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");

    job_count++;
}

void update_jobs()
{
    ^G Help      ^O Write Out
    ^X Exit      ^R Read File
               ^W Where Is
               ^\ Replace
```

```
for (int i = 0; i < job_count; i++)
{
    if (strcmp(jobs[i].status, "Running") == 0)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status))
            {
                strcpy(jobs[i].status, "Done");
            }
            else
            {
                strcpy(jobs[i].status, "Terminated");
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("==== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

```
GNU nano 7.2
    "Command");

    printf("-----\n");
    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].status,
                jobs[i].command);
    }
    if (job_count == 0)
    {
        printf("No jobs found.\n");
    }
    printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return;
    }
    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
                getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
                getpid());
    }
}
```

GNU nano 7.2
^G Help **^O** Write Out **^W** Where Is **^K** Cut
^X Exit **^R** Read File **^_** Replace **^U** Paste

```
GNU nano 7.2
printf("==== JOB MANAGEMENT SYSTEM =====\n");

while (1)
{
    printf("\n");
    printf("1. Start Background Job\n");
    printf("2. List Jobs\n");
    printf("3. Exit\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("Enter job duration in seconds: ");
        scanf("%d", &seconds);
        start_job(seconds);
    }
    else if (choice == 2)
    {
        list_jobs();
    }
    else if (choice == 3)
    {
        update_jobs();
        printf("\nExiting job manager...\n");
        break;
    }
    else
    {
        printf("Invalid choice.\n");
    }
}

return 0;
}
```

GNU nano 7.2
^G Help **^O** Write Out **^W** Where Is **^K** Cut
^X Exit **^R** Read File **^_** Replace **^U** Paste

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir job_practical
jhanasri@LAPTOP-1RN1M8E3:~$ cd job_practical
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ pwd
/home/jhanasri/job_practical
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ ls
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ nano jobs.c
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ ls
jobs.c
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ cat jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");
}

void update_jobs
```

```

{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].status, "Running") == 0)
        {
            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result == jobs[i].pid)
            {
                if (WIFEXITED(status))
                {
                    strcpy(jobs[i].status, "Done");
                }
                else
                {
                    strcpy(jobs[i].status, "Terminated");
                }
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\n");
    printf("==== ACTIVE JOBS =====\n");

    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");

    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,

```

```

    {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
    }

    if (job_count == 0)
    {
        printf("No jobs found.\n");
    }

    printf("-----\n");
}

void start_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n",
            getpid());

        sleep(seconds);

        printf("Child process %d completed.\n",
            getpid());

        exit(0);
    }
    else
    {
        char command[100];

        snprintf(command,
            sizeof(command),
            "sleep %d",
            seconds);
    }
}

```

```

        add_job(pid, command);

        printf("Job [%d] started with PID %d\n",
               job_count,
               pid);
    }
}

int main()
{
    int choice;
    int seconds;

    printf("==== JOB MANAGEMENT SYSTEM ====\n");

    while (1)
    {
        printf("\n");
        printf("1. Start Background Job\n");
        printf("2. List Jobs\n");
        printf("3. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Enter job duration in seconds: ");
            scanf("%d", &seconds);

            start_job(seconds);
        }
        else if (choice == 2)
        {
            list_jobs();
        }
        else if (choice == 3)
        {
            update_jobs();

            printf("\nExiting job manager...\n");
            break;
        }
        else
        {
            printf("Invalid choice.\n");

```

```

1. Start Background Job
2. List Jobs
3. Exit
Enter choice: Child process started. PID = 580
2

===== ACTIVE JOBS =====
Job ID   PID      Status      Command
-----
1         573      Done        sleep 5
2         580      Running     sleep 10
-----

1. Start Background Job
2. List Jobs
3. Exit
Enter choice: Child process 580 completed.
2

===== ACTIVE JOBS =====
Job ID   PID      Status      Command
-----
1         573      Done        sleep 5
2         580      Done        sleep 10
-----

1. Start Background Job
2. List Jobs
3. Exit
Enter choice: 9
Invalid choice.

1. Start Background Job
2. List Jobs
3. Exit
Enter choice: 3

Exiting job manager...
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ ls -l
total 24
-rwxr-xr-x 1 jhanasri jhanasri 16704 Sep 19 08:45 jobs
-rw-r--r-- 1 jhanasri jhanasri 3321 Sep 19 08:45 jobs.c
jhanasri@LAPTOP-1RN1M8E3:~/job_practical$ |

```

2 . To Design how to Identify Target Job, Transfer Terminal

Control, Resume Processes, Wait for Completion, Update Job State, and Test Foreground Switching.

Aim : To implement foreground job control by identifying a target job, transferring terminal control to the selected process, resuming a stopped process, waiting for its completion, updating the job state, and testing foreground switching.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <termios.h>

int main()
{
    pid_t pid;
    int status;
    int shell_terminal;

    shell_terminal = STDIN_FILENO;

    printf("===== FOREGROUND JOB CONTROL =====\n");

    /* Create a child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        printf("\nChild process created.\n");
        printf("Child PID: %d\n", getpid());

        /* Create a new process group */
        setpgid(0, 0);

        /* Resume process if it was stopped */
        kill(getpid(), SIGCONT);

        printf("Child process is running in foreground...\n");

        for (int i = 1; i <= 5; i++)
        {
            printf("Child working... %d/5\n", i);
            sleep(1);
        }

        printf("Child process completed.\n");
        exit(0);
    }
}
```

```

GNU nano 7.2

/* Parent process */

printf("\nParent process created child.\n");
printf("Child PID: %d\n", pid);

/* Put child in its own process group */
setpgid(pid, pid);

printf("\nIdentifying target job...\n");
printf("Target Job PID: %d\n", pid);

printf("\nTransferring terminal control to child...\n");

/*
 * Give terminal control to child process group.
 */
if (tcsetpgrp(shell_terminal, pid) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control transferred.\n");

/* Wait for child */
printf("\nParent is waiting for foreground job...\n");
if (waitpid(pid, &status, WUNTRACED) == -1)
{
    perror("waitpid failed");
    return 1;
}

/* Check job state */
if (WIFSTOPPED(status))
{
    printf("\nJob stopped.\n");
    printf("Resuming foreground job...\n");
    kill(pid, SIGCONT);
    waitpid(pid, &status, 0);
}

if (WIFEXITED(status))
{
    printf("\nForeground job completed normally.\n");
    printf("Exit status: %d\n",
        WEXITSTATUS(status));
}

}

/* Give terminal back to parent */
printf("\nReturning terminal control to parent...\n");

if (tcsetpgrp(shell_terminal, getpgrp()) == -1)
{
    perror("tcsetpgrp");
}

printf("Terminal control returned to parent.\n");

printf("\nJob state updated: COMPLETED\n");

printf("\n==== FOREGROUND SWITCHING TEST COMPLETED =====\n");

return 0;
}

```

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~$ mkdir foreground_job
jhanasri@LAPTOP-1RN1M8E3:~$ cd foreground_job
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ pwd
/home/jhanasri/foreground_job
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ ls
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ nano foreground_job.c
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ gcc foreground_job.c -o foreground_job
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ ls
foreground_job  foreground_job.c
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ ./foreground_job
===== FOREGROUND JOB CONTROL =====

Parent process created child.

Child process created.
Child PID: 761
Child PID: 761

Identifying target job...
Target job PID: 761

Transferring terminal control to child...
Terminal control transferred.

Child process is running in foreground...
Parent is waiting for foreground job...
Child working... 1/5
Child working... 2/5
Child working... 3/5
Child working... 4/5
Child working... 5/5
Child process completed.

Foreground job completed normally.
Exit status: 0

Returning terminal control to parent...

[1]+  Stopped                  ./foreground_job
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ ls -l
total 24
-rwxr-xr-x 1 jhanasri jhanasri 3686 Sep 19 08:14 foreground_job
-rw-r--r-- 1 jhanasri jhanasri 3066 Sep 19 08:13 foreground_job.c
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$ file foreground_job
foreground_job: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-
linux-x86-64.so.2, BuildID[sha1]=963686f73be63da7
5a4f62b7f99886c62f2061b, for GNU/Linux 3.2.0, not stripped
jhanasri@LAPTOP-1RN1M8E3:~/foreground_job$

```

3 . To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior using a C program and Linux commands.

Aim : To study how signals are delivered to processes, how signal masks block and unblock signals, how process groups are identified, and how terminal signals such as **SIGINT** and **SIGTSTP** affect processes.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received: Ctrl+C signal delivered.\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received: User-defined signal delivered.\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received: Termination signal delivered.\n");
}

int main()
{
    sigset_t mask;
    pid_t pid = getpid();

    printf("Signal Behavior Analysis\n");
    printf("-----\n");

    printf("Process PID      : %d\n", pid);
    printf("Process Group ID : %d\n", getpgrp());
    printf("Parent PID       : %d\n", getppid());

    /* Register signal handlers */
    signal(SIGINT, signal_handler);
    signal(SIGUSR1, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("\nSignal handlers registered.\n");
}
```

Output :

```

jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_sigtstp.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_sigtstp.c -o task_sigtstp
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_sigtstp
Shell Process Started
Shell PID: 834

Shell: Foreground job PID = 835
Shell: Job State = RUNNING
Press Ctrl+Z to suspend the job.

Foreground job started.
Job PID: 835
Job State: RUNNING
Job is working...
Foreground job is running...
Foreground job is running...
Foreground job is running...
^Z
Shell: SIGTSTP received.
Shell: Suspending foreground job (PID 835)
Shell: Job State = STOPPED
Shell: Job state preserved.

Shell: Job PID 835 has been suspended.
Shell: Job State = STOPPED

Press ENTER to resume the job.
Enter
Shell: Sending SIGCONT to job PID 835
Shell: Job State = RUNNING
Shell: Job resumed successfully.
Foreground job is running...
Foreground job is running...
Foreground job is running...

Shell: Terminating foreground job.
Shell: Job State = TERMINATED
Shell: Job removed from foreground.
Shell: Process completed.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ |

```

4 . To implement resuming of stopped jobs by sending continue signals, updating job status, maintaining a job table, verifying execution, and testing background recovery.

Aim : To demonstrate how a stopped process can be resumed using **SIGCONT**, update its job status from STOPPED to RUNNING, and manage the job using a simple job table.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t shell_pgid;
pid_t child_pgid;

/* Handle Ctrl+C */
void handle_sigint(int sig)
{
    (void)sig;

    printf("\nShell: SIGINT received.\n");

    if (child_pgid > 0) {
        printf("Shell: Forwarding SIGINT to process group %d\n",
            child_pgid);

        kill(-child_pgid, SIGINT);
    }
}

int main()
{
    pid_t pid;
    int status;

    shell_pgid = getpgrp();

    printf("Shell Process Started\n");
    printf("Shell PID      : %d\n", getpid());
    printf("Shell Group ID: %d\n", shell_pgid);

    /* Register SIGINT handler */
    signal(SIGINT, handle_sigint);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
```

^G Help	^O Write Out	^W Where Is	^K Cut
^X Exit	^R Read File	^\ Replace	^U Paste

```
/* Parent */
child_pgid = pid;

/* Make sure child is in its own process group */
setpgid(pid, pid);

printf("\nShell: Child created.\n");
printf("Shell: Child PID      = %d\n", pid);
printf("Shell: Child Group ID = %d\n", getpgid(pid));

/*
 * Transfer terminal control to child process group.
 * tcsetpgrp() may fail when the program is not running
 * with a controlling terminal, so report the error.
 */
if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1) {
    perror("tcsetpgrp - transfer terminal");
} else {
    printf("Shell: Terminal control transferred to child group.\n");
}

sleep(3);

printf("\nShell: Sending SIGINT to child process group.\n");
kill(-child_pgid, SIGINT);

sleep(2);

/* Restore terminal control to shell */
if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
    perror("tcsetpgrp - restore terminal");
} else {
    printf("Shell: Terminal control restored.\n");
}

/* Terminate child */
kill(-child_pgid, SIGTERM);

waitpid(pid, &status, 0);

printf("Shell: Child process terminated.\n");
printf("Shell: Process group management completed.\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T Exec
^J Just

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ nano task_process_groups.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ gcc -Wall -Wextra task_process_groups.c -o task_process_groups
jhanasri@LAPTOP-1RN1M8E3:~/OSSP$ ./task_process_groups

Shell Process Started
Shell PID      : 1045
Shell Group ID: 1045

Shell: Child created.
Shell: Child PID      = 1046
Shell: Child Group ID = 1046
Shell: Terminal control transferred to child group.

Child Process Started
Child PID      : 1046
Child Group ID: 1046
Child: Waiting for signals...
```

[illegible]

```
jhanasri@LAPTOP-1RN1M8E3:~$ ps -ef | grep task_process_groups
jhanasri      888      1 89 05:33 ?          00:07:39  ./task_process_groups
jhanasri     1005      1 88 05:38 ?          00:02:50  ./task_process_groups
jhanasri     1045     1016  0 05:40 pts/3      00:00:00  ./task_process_groups
jhanasri     1046     1045  4 05:40 pts/3      00:00:05  ./task_process_groups
jhanasri     1076     1057  0 05:42 pts/4      00:00:00  grep --color=auto task_process_groups
jhanasri@LAPTOP-1RN1M8E3:~$ kill -9 1046
jhanasri@LAPTOP-1RN1M8E3:~$ |
```

Observation

The `ps -ef | grep task_process_groups` command displays the running `task_process_groups` processes along with their **PID, PPID, process group, terminal, CPU time, and command**. The command `kill -9 1046` is then used to forcefully terminate the child process with PID **1046**.

Result

The process with PID 1046 is terminated successfully. The terminal returns to the shell prompt, indicating that the process has been removed from execution.